SharePoint Online Permissions & Access Management

Implementation Solution Guide

Version 1.0.0 · 2026-07-07

Contents

Executive Summary 4

The outcome . . . 4

The approach . . . 4

Why it is low-risk . . . 4

What's delivered . . . 4

1. Solution Overview — SharePoint Online Permissions & Access Management 5

What this solution is . . . 5

Scope — what this is, and what it deliberately is NOT . . . 5

Why this matters . . . 5

How it works . . . 6

The standard permission-level model . . . 7

The reference model (artifacts this solution creates) . . . 7

Benefits . . . 8

Limitations (be honest with buyers) . . . 8

The standard deployment model . . . 9

2. Requirements 9

1. SharePoint Administrator (and a site-collection admin path) . . . 9

2. PnP PowerShell with a registered Entra app (for scoped audit/remediation) . . . 10

3. A defined group source of truth (Entra/M365 groups vs SharePoint groups) . . . 10

4. Information owners / approvers per site (the entitlement decision) . . . 10

5. Licensing for advanced review/reporting (Entra ID P2 and/or SAM) — verify . . . 11

6. Microsoft Graph permissions for reporting (where Graph is used) . . . 11

7. A pilot scope and documented rollback . . . 11

Global Requirements — SharePoint Online & OneDrive Solutions 12

1. Microsoft 365 tenant with SharePoint Online / OneDrive and licensing . . . 12

2. Administrative access (least-privilege SharePoint roles) . . . 12

3. SharePoint Online Management Shell / PnP + Graph toolchain . . . 13

4. Information architecture & identity/group source of truth . . . 13

5. A pilot scope and change control . . . 13

6. Security, compliance & records stakeholders . . . 13

3. Implementation 14

Stage A — Assess (read-only): baseline the permission posture . . . 14

Stage B — Design: define the group + permission-level model . . . 15

Stage C — Configure: stand up group-based access . . . 15

Stage D — Pilot remediation (`-WhatIf` first, then `-Commit`) . . . 16

Stage E — Validate the pilot . . . 17
Stage F — Stand up access reviews, then expand . . . 17
5. Testing & Validation 18
Test 1 — Access is group-based at the site root . . . 18
Test 2 — Standard permission levels are correctly bound . . . 19
Test 3 — Broken inheritance reduced to documented exceptions only . . . 19
Test 4 — Limited Access sprawl reduced . . . 19
Test 5 — No unintended over-share . . . 19
Test 6 — A converted user actually has the right effective access . . . 19
Test 7 — Access review is running . . . 20
Test 8 — The access report reproduces cleanly . . . 20
Test 9 — Rollback works (pilot only, non-destructive rehearsal) . . . 20
6. Troubleshooting 21
Connection & permission problems . . . 21
Audit / discovery problems . . . 21
Remediation problems . . . 22
Limited Access & inheritance quirks . . . 22
Ongoing model / drift problems . . . 23
7. Best Practices 24
Model & structure . . . 24
Inheritance & unique permissions . . . 24
Change discipline . . . 24
Ongoing attestation & drift control . . . 25
Cloud & licensing awareness . . . 25
8. Deliverables Summary 25
Documentation . . . 25
PowerShell scripts (5) — SharePoint Online Management Shell / PnP PowerShell / Microsoft Graph 26
Appendix A — PowerShell Scripts 29

Executive Summary

The outcome

SharePoint Online access becomes **intentional and auditable**. Instead of a tangle of broken inheritance, item-level unique permissions, untraceable direct-to-user grants, and over-shared sites, access flows through a small set of **named groups** with **standard permission levels**, backed by the organization's identity source of truth. "Who can see this site?" becomes a one-line answer — the membership of a named group — and it stays that way because recurring reviews and reporting catch drift.

The approach

The solution follows a disciplined, evidence-first path: **Assess** → **Design** → **Configure** → **Pilot** → **Validate** → **Expand**. It begins with a **read-only audit** that inventories every broken-inheritance scope, unique permission, direct user grant, Limited Access entry, and over-share signal — the baseline the information owner reviews and the record we can roll back to. It then defines a **group-based access model** and a **standard permission-level model**, converts direct grants into group membership, and **resets broken inheritance back to inherit** everywhere except a small set of documented, owner-approved exceptions. Finally it stands up **recurring access reviews** and **repeatable access reporting** so the model holds.

Why it is low-risk

- **Read-only first.** Nothing changes until a full audit exists and the site owner has reviewed each exception. The audit is also the rollback baseline.
- **Safe-by-default automation.** Every remediation script defaults to **dry-run /** `-WhatIf`; only an explicit `-Commit` writes. No credentials or secrets live in the code — modern-auth sign-in only.
- **Pilot-first with pass/fail gates.** The entire cycle runs on a small pilot site and must pass explicit validation gates before expanding, with a documented rollback at every step.
- **Business decisions stay with the business.** The solution supplies the technical model, the remediation method, and the evidence; **who belongs in each group** remains an information-owner decision, and licensing-gated platform features (Entra ID P2 Access Reviews, SharePoint Advanced Management reports) are used where available or cleanly substituted with included PowerShell reporting.

What's delivered

A full documentation set (overview, requirements, step-by-step implementation, testing, troubleshooting, best practices) plus **five PowerShell tools** — a read-only permissions/over-share audit, a group-based access remediation helper, a broken-inheritance reset helper, an access-review/reporting tool, and a scope/health checker — all safe-by-default and authentic to the SharePoint Online Management Shell / PnP PowerShell / Microsoft Graph toolchain. Self-guided and tested, at a fraction of the cost of a consulting engagement.

1. Solution Overview — SharePoint Online Permissions & Access Management

What this solution is

SharePoint permissions rot. What starts as a clean, inheriting site collection turns — over months of "just give Jane access to this one folder" — into a tangle of **broken inheritance**, thousands of **unique permissions**, **direct user grants** nobody can trace, and sites shared so widely that no one can answer "who can see this?". This solution establishes a durable **permissions and access model** for SharePoint Online and remediates the existing mess: it standardizes on **group-based access** (SharePoint groups and Entra ID / Microsoft 365 groups instead of direct user grants), defines a **standard permission-level model**, **controls unique permissions and broken inheritance** (resetting them to inherit where they should, keeping them only where genuinely required), stands up **site-access reviews**, and delivers **access reporting** so who-can-see-what is intentional and auditable.

The work is done through the **SharePoint admin center**, the **Microsoft 365 admin center**, the **Microsoft Entra admin center** (for M365/Entra groups and Access Reviews), and **PowerShell** — the **SharePoint Online Management Shell**, **PnP PowerShell**, and **Microsoft Graph** — with everything **pilot-first** and safe-by-default.

Scope — what this is, and what it deliberately is NOT

This solution is **scoped to the SharePoint Online permissions and access model**: SharePoint groups vs Entra/M365 groups, standard permission levels, inheritance vs unique permissions, **Limited Access** sprawl, direct-grant remediation, site-access reviews, and access reporting.

It is **not**:

- **External / guest sharing governance** — anonymous "Anyone" links, guest access, and tenant/site sharing controls are a **separate concern covered by a different product**. This solution governs the *internal* permission model (who inside the org has what level, via which group); it references the sharing boundary but does not configure external-sharing policy.
- **Information architecture / site provisioning** — the hub/site structure and the *provisioning* templates that create sites with the right groups are a separate product. This solution assumes the IA exists (see Requirements) and governs permissions *within* it.
- **Records, retention, or sensitivity labels** — Purview retention/records and sensitivity labels are a separate data-governance concern. Permissions control *access*; labels/retention control *lifecycle*.
- **A compliance or legal determination.** Which business roles *should* have access to which content is an information-owner/compliance decision. This solution provides the **technical model, the remediation method, and the evidence** (audit + reports); the org's information owners decide who belongs in each group.

Why this matters

- **Unauditable access is a risk and an audit finding.** When access is granted directly to individual users on individual items, there is no single place to answer "who can see the board deck?" Group-based access makes access legible: membership of a named group *is* the answer.
- **Broken inheritance is unmaintainable.** Every place inheritance is broken becomes a permission boundary that must be maintained by hand forever. Thousands of them across a tenant means access drifts, removals get missed, and offboarded users retain access on scoped items.
- **Limited Access sprawl hides real exposure.** Granting a user access to one document silently gives them **Limited Access** at the parent list and site — a real permission level SharePoint adds automatically. Enough of these and the site's permissions page becomes noise, masking who actually has meaningful access.
- **Direct grants outlive people and roles.** A user added directly to a site stays there after they change teams; a user added to the `SPO Group – Finance Members – Prod` group is removed the moment they leave the group. Group-based access is self-cleaning; direct grants are not.

How it works

1. **Assess first (read-only).** Run the discovery audit across the in-scope sites to inventory **broken inheritance**, **unique permissions**, **direct user grants**, **Limited Access** entries, and **over-shared** scopes (e.g. "Everyone except external users" on a whole site). This produces the evidence baseline `Permission Audit – SPO – Discovery` — no changes are made.
1. **Design the model.** Define the `Access Model – SPO – Design`: which **SharePoint groups** and **Entra/M365 groups** back each site, the **standard permission-level model** (below), and the rule for when unique permissions are *allowed* (rare, documented, owner-approved) vs *removed* (the default).
1. **Configure the group-based model.** For each in-scope site, ensure access flows through the site's SharePoint groups (Owners/Members/Visitors) whose membership is, wherever possible, an **Entra/M365 group** — not a list of individual users. Assign the standard permission levels to those groups.
1. **Remediate (pilot-first, `-WhatIf` by default).** Convert **direct user grants** into group membership, then **reset broken inheritance** back to inherit where the design says it should — keeping unique permissions only on the documented exceptions. Every reset and conversion is a named change: `Inheritance Reset – <Site> – Remediation`, `Group Access Remediation – <Site> – Remediation`.
1. **Stand up access reviews.** Wire **recurring site-access reviews** so site owners (or an approver) periodically re-attest who should keep access — via **Microsoft Entra ID Access Reviews** (P2-licensed) for group membership, and/or SharePoint's **site access review** where **SharePoint Advanced Management (SAM)** is licensed. Named `Access Review – <Site> – Prod`.
1. **Report continuously.** Deliver the `Access Report – SPO – Report` — a repeatable who-has-access export per site — plus the SharePoint admin center **Data Access Governance (DAG)** reports where SAM is licensed, so drift is caught between reviews.

The standard permission-level model

This solution standardizes on SharePoint's **built-in permission levels** and adds custom levels only when a real need exists. The built-in levels used:

Level	Grants (built-in)	Typical group
Full Control	Everything, incl. permissions management	Site **Owners** group only
Edit	Add/edit/delete lists; add/edit/delete list items & documents	Site **Members** (contributors)
Contribute	Add/edit/delete list items & documents (not lists themselves)	Restricted contributors
Read	View pages, list items, and documents; download	Site **Visitors** (read-only)
Limited Access	*Special/automatic* — access to a specific item, not the site. **Not assigned by hand**; SharePoint adds it when unique permissions are granted below the site	*(system-managed)*

> ***Custom levels are real but used sparingly.*** *A common custom level is a view-only level that removes the *download / copy* permission (`Perm Level – Read No Download – Prod`). Custom permission levels are legitimate, but each one is another thing to maintain — define them only when the built-ins can't express the requirement, and never edit the **built-in** levels in place. **View Only** (as used by Office Online / library "Open in browser only") behaves differently from Read; verify the exact effect in your tenant before relying on it.*

The reference model (artifacts this solution creates)

Every artifact is named `<Purpose> – <Scope> – <Stage>` and reused verbatim across the docs, scripts, admin center, and reports:

Artifact	Name
Read-only permission audit / baseline	`Permission Audit – SPO – Discovery`
The group + permission-level model doc	`Access Model – SPO – Design`
Entra/M365 group backing a site's members	`M365 Group – Finance – Prod`
SharePoint group (Members) for a site	`SPO Group – Finance Members – Prod`
SharePoint group (Owners) for a site	`SPO Group – Finance Owners – Prod`
SharePoint group (Visitors) for a site	`SPO Group – Finance Visitors – Prod`
Optional custom view-only level	`Perm Level – Read No Download – Prod`
Pilot remediation scope	`Permission Scope – Pilot – Stage1`

Artifact	Name
Broad remediation scope	`Permission Scope – Broad – Stage2`
A broken-inheritance reset (per site/scope)	`Inheritance Reset – <Site> – Remediation`
A direct-grant→group conversion	`Group Access Remediation – <Site> – Remediation`
A recurring site/group access review	`Access Review – <Site> – Prod`
The repeatable access report	`Access Report – SPO – Report`

> *Replace `Finance` / `<Site>` with your real site name and keep the pattern. Never invent a site or owner name.*

Benefits

• **Access becomes legible.** "Who can see this site?" is answered by naming a group's membership, not by

reading thousands of item-level ACLs.

• **Permissions become self-maintaining.** Membership changes (joins/leaves/transfers) flow through

Entra/M365 groups; access follows automatically. No orphaned direct grants.

• **The existing mess is remediated safely.** Broken inheritance and Limited Access sprawl are reset back

to inherit — pilot-first, dry-run-first, with a rollback — so nothing breaks.

• **Access is continuously attested.** Recurring reviews force periodic re-confirmation; DAG/report exports

catch drift in between.

Limitations (be honest with buyers)

• **Some review/reporting features are license-gated. Entra ID Access Reviews** require **Entra ID

P2 **(or an equivalent suite). SharePoint's** Data Access Governance reports **and** site access review **require** SharePoint Advanced Management (SAM)**. Where those aren't licensed, this solution falls back to the included PowerShell reporting —** verify in your tenant/plan**.

• **Resetting inheritance is a real, visible change.** Resetting a scope to inherit *removes* the unique

permissions there — which is the point, but if a unique grant was load-bearing and undocumented, someone loses access. That is why remediation is **pilot-first,** `-WhatIf` **first**, with the audit as the before-state and a documented rollback.

• **Limited Access cannot be assigned or removed directly.** It is system-managed. You remove it by

removing the *underlying* unique grant that caused it, not by editing Limited Access itself.

• **Group-based access needs a group source of truth.** If the org has no coherent Entra/M365 group model,

the group-based model rests on SharePoint groups alone (still an improvement) until the group model is established (often the Architecture product).

- **Who belongs in each group is an information-owner decision.** This solution builds the model and the

evidence; it does not decide business entitlement.

- **Commercial vs Government clouds differ.** Admin-center URLs and some capability availability differ on

GCC / GCC High / DoD (`*.sharepoint.us`, `*.office365.us`, `portal.azure.us`) — noted where it matters; verify on your cloud.

The standard deployment model

> *Assess the current permission posture (audit) → design the group + permission-level model → configure group-based access → remediate direct grants and broken inheritance on a **pilot** scope first → validate (access is group-based, inheritance intact where intended, over-share reduced, reviews running) → expand to the broad scope. Pattern: **Assess** → **Design** → **Configure** → **Pilot** → **Validate** → **Expand**, with explicit pass/fail gates and a documented rollback.*

2. Requirements

These are the **solution-specific** prerequisites for SharePoint Online Permissions & Access Management. They build **on top of** the shared **Global Requirements — SharePoint Online & OneDrive Solutions** (`global-requirements-sharepoint.md`): a licensed M365 tenant with SharePoint Online, least-privilege admin roles, the SPO Management Shell / PnP / Graph toolchain, a defined information architecture and identity/group source of truth, a pilot scope with change control, and engaged security/compliance stakeholders. Verify those first; the items below are what this solution needs **in addition**.

Each requirement is stated as **Description / Why / Verify / Implement if missing**.

1. SharePoint Administrator (and a site-collection admin path)

- **Description:** A **SharePoint Administrator** role to run tenant-level audit/reporting, plus a way to

be **Site Collection Administrator** on each in-scope site (SharePoint Admin can grant itself SCA on a site, or the site owner adds it). Global Administrator is **not** required for permission remediation.

- **Why required:** Reading and resetting role assignments, resetting inheritance, and managing site

groups requires SCA on the target site; tenant audit/reporting requires SharePoint Admin.

- **How to verify:** In the M365 admin center confirm the **SharePoint Administrator** role; confirm

`Connect-SPOService`/`Connect-PnPOnline` succeeds and that you can read a site's groups (`Get-PnPGroup`).

- **How to implement if missing:** Have a Global Admin assign **SharePoint Administrator** under change

control; grant SCA on the pilot sites (SharePoint admin center → *Sites → Active sites → <site> → Membership*, or `Set-SPOUser -IsSiteCollectionAdmin $true`).

2. PnP PowerShell with a registered Entra app (for scoped audit/remediation)

• **Description: PnP PowerShell** (`PnP.PowerShell`) installed, with an **Entra ID app registration** for interactive/delegated modern-auth sign-in. As of PnP.PowerShell v2+, `Connect-PnPOnline -Interactive` requires an app registration (`Register-PnPManagementShellAccess` or a custom app) with delegated SharePoint permissions.

• **Why required:** The discovery, remediation, review-support, and reporting scripts run on PnP against each site; item/list-level inheritance and role-assignment operations are PnP/CSOM.

• **How to verify:** `Get-Module PnP.PowerShell -ListAvailable`; `Connect-PnPOnline -Url https://contoso.sharepoint.com/sites/<pilot> -Interactive succeeds and Get-PnPWeb` returns the web.

• **How to implement if missing:** `Install-Module PnP.PowerShell`; register the app (`Register-PnPManagementShellAccess`) or use your org's existing PnP app; consent the delegated permissions. **No client secret is stored in the scripts** — modern-auth interactive/delegated only.

3. A defined group source of truth (Entra/M365 groups vs SharePoint groups)

• **Description:** A decision on **what backs access**: Entra ID security groups / Microsoft 365 groups as the authoritative membership, surfaced into each site via its **SharePoint groups** (Owners / Members / Visitors). Team sites created from a Microsoft 365 group already have the group as the owner/member source; communication sites and classic sites use SharePoint groups directly.

• **Why required:** Group-based access is only "self-maintaining" if membership lives in a governed group.

Without a source of truth, you can still standardize on SharePoint groups, but joins/leaves won't flow automatically.

• **How to verify:** Confirm whether in-scope sites are **M365-group-connected** (Team sites) or not; confirm which Entra/M365 groups are authoritative for each business area.

• **How to implement if missing:** Define the group model with the identity team (often the Architecture product). At minimum, agree that access is granted to **SharePoint groups**, never to individual users.

4. Information owners / approvers per site (the entitlement decision)

• **Description:** A named **information owner or approver** for each in-scope site who can decide **who should have access **and at what level, and who will** run the access reviews**.

• **Why required:** This solution builds the model and surfaces the evidence, but the **business

entitlement** — who belongs in `SPO Group – Finance Members – Prod` — is an owner decision, not the implementer's. Remediation that removes a unique grant needs an owner to confirm it was not intended.

• **How to verify:** Confirm a named owner/approver exists for each pilot site and agrees to review the audit exceptions.

• **How to implement if missing:** Identify the owner with the business/compliance stakeholders before remediating; capture their decisions against the `Permission Audit – SPO – Discovery` exception list.

5. Licensing for advanced review/reporting (Entra ID P2 and/or SAM) — verify

• **Description: Microsoft Entra ID P2** (for **Access Reviews**) and/or **SharePoint Advanced Management (SAM) **(for** Data Access Governance reports **and** site access review**) if you intend to use those platform features rather than the included PowerShell reporting.

• **Why required:** Access Reviews and DAG reports are **license-gated add-ons**; availability and the exact feature set are **plan-dependent and change over time**.

• **How to verify:** In the M365 admin center / Entra admin center confirm **Entra ID P2** assignment; in the SharePoint admin center confirm the **Data Access Governance** and **access review** features are present (they surface only when SAM is licensed). **Verify in your tenant/plan** — do not assume.

• **How to implement if missing:** Either license the add-ons under change control, **or** use the solution's included PowerShell reporting/review-support scripts, which cover the same evidence without the platform features. The solution works either way.

6. Microsoft Graph permissions for reporting (where Graph is used)

• **Description:** For the Graph-based reporting paths, delegated Microsoft Graph scopes such as `Sites.Read.All`, `Group.Read.All`, and `Directory.Read.All` (read-only), consented for the sign-in account.

• **Why required:** Expanding Entra/M365 group membership and cross-site reporting uses Graph.

• **How to verify:** `Connect-MgGraph -Scopes 'Sites.Read.All','Group.Read.All','Directory.Read.All'` succeeds and `Get-MgGroup -Top 1` returns.

• **How to implement if missing:** Have an admin consent the delegated read scopes; **no application secret is stored** — modern-auth delegated sign-in only.

7. A pilot scope and documented rollback

• **Description:** A **named pilot site (or small set)** to remediate first — ideally one with real

broken inheritance / direct grants but limited blast radius — and a **rollback** approach (the audit export is the before-state; re-granting a removed unique permission is the reverse action).

- **Why required:** Resetting inheritance and converting direct grants are **high-impact** changes.

Pilot-first with a rollback is the house model.

- **How to verify:** Confirm the pilot site (`Permission Scope – Pilot – Stage1`), the change window, and

that a fresh `Permission Audit – SPO – Discovery` export has been captured for it.

- **How to implement if missing:** Choose the pilot site, capture the audit baseline, and agree the change

window/rollback with the site owner before any `-Commit` run.

Global Requirements — SharePoint Online & OneDrive Solutions

These requirements apply to **every** solution in the **SharePoint** category (SharePoint Online architecture & provisioning, external sharing, OneDrive for Business, migration to SPO, permissions, search, and records/retention). Verify each before starting a solution's implementation steps.

> ***Vendor boundary (applies to every SharePoint solution).*** *The **SharePoint admin center**, the **Microsoft 365 admin center**, the **SharePoint Online Management Shell** and **PnP PowerShell**, and **Microsoft Graph** are authored at exact, current click-by-click / cmdlet detail from Microsoft's public documentation. Where a capability, setting, or portal path is **plan- or version-dependent** (storage limits, sharing controls, Syntex/records features, Commercial vs Government cloud), it is **cited or flagged "verify in your tenant"** — never invented. **Third-party migration and governance tools** (e.g. ShareGate, Migration Manager, AvePoint) are integrated at the pattern level; their consoles are deferred to the vendor's docs. Automation is **PowerShell** (SPO Management Shell / PnP / Graph) — never with hardcoded credentials or secrets.*

1. Microsoft 365 tenant with SharePoint Online / OneDrive and licensing

- **Description:** A Microsoft 365 tenant with **SharePoint Online** and **OneDrive for Business** and

the licensing the capability requires (e.g. an E-plan for advanced records/retention, Syntex where used).

- **Why required:** All site, sharing, OneDrive, and records configuration depends on the right plan.
- **How to verify:** In the M365 admin center confirm the SharePoint/OneDrive subscription and add-ons.
- **How to implement if missing:** Assign/purchase the required licensing before starting.

2. Administrative access (least-privilege SharePoint roles)

- **Description:** Access with the least-privilege role the solution names — typically **SharePoint

Administrator** (and Global Administrator only where genuinely required, e.g. tenant-wide sharing).

- **Why required:** Tenant/site settings, sharing, provisioning, and records changes require the role.

- **How to verify:** Confirm the role in the M365 admin center / Entra roles.
- **How to implement if missing:** Have a Global Admin assign the least-privilege role under change control.

3. SharePoint Online Management Shell / PnP + Graph toolchain

- **Description:** A workstation with the **SharePoint Online Management Shell** and/or **PnP PowerShell** (and Graph PowerShell where used), modern-auth / MFA-capable admin sign-in (PnP app registration where required).
- **Why required:** Configuration, automation, and the provided scripts run through SPO/PnP/Graph.
- **How to verify:** `Connect-SPOService` / `Connect-PnPOnline` succeeds; modules installed.
- **How to implement if missing:** Install the modules; register the PnP app where needed; confirm sign-in.

4. Information architecture & identity/group source of truth

- **Description:** A defined **information-architecture** intent (hub/site structure, taxonomy) and the identity/group source (Entra ID groups, M365 Groups backing team sites) that drives access.
- **Why required:** Provisioning, permissions, search, and governance key off the IA and groups.
- **How to verify:** Confirm the intended hub/site structure and which groups model is authoritative.
- **How to implement if missing:** Define the IA and group model (often the Architecture solution) first.

5. A pilot scope and change control

- **Description:** A **pilot site collection / hub / user set** to validate on before tenant-wide rollout, and a **change-control** process for production changes (sharing, permissions, retention, migration cutover are high-impact).
- **Why required:** Every solution follows an **Assess** → **Design** → **Configure** → **Pilot** → **Validate** → **Expand** model; tenant-wide sharing/permission/retention changes are high-impact.
- **How to verify:** Confirm a named pilot scope and an approval/change window are available.
- **How to implement if missing:** Create a pilot scope and agree change control before production changes.

6. Security, compliance & records stakeholders

- **Description:** Engagement with the **security, compliance/records, and data-governance** stakeholders whose policies the configuration must satisfy (external sharing, permissions, retention/records, sensitivity).
- **Why required:** Sharing, permission, and records settings must align to enterprise policy.

• **How to verify:** Confirm the applicable policies and named approvers.

• **How to implement if missing:** Identify the owners and capture requirements before configuring governance.

*The deployment model across every SharePoint solution is the same: **Assess** → **Design** → **Configure** → **Pilot** → **Validate** → **Expand**, with explicit pass/fail gates and a documented rollback for production changes. Automation is delivered as **PowerShell** (SPO Management Shell / PnP / Graph) — never with hardcoded credentials or secrets in code.*

3. Implementation

Follow these steps in order. They implement the house model **Assess** → **Design** → **Configure** → **Pilot** → **Validate** → **Expand**, pilot-first, with explicit **pass/fail gates** and a **rollback**. Every artifact is named `<Purpose> – <Scope> – <Stage>` and reused verbatim in later steps, testing, troubleshooting, and the scripts. Examples use `contoso.sharepoint.com` and a `Finance` site — **substitute your tenant and real site names**; never invent an org.

> ***Safety.*** *Every change is preceded by a read-only audit and runs* `-WhatIf` ***/ dry-run by default****. Only an explicit* `-Commit` *writes. Do the whole cycle on* `Permission Scope – Pilot – Stage1` *before touching* `Permission Scope – Broad – Stage2`*.*

Stage A — Assess (read-only): baseline the permission posture

A1. Connect with least privilege

1. Install/confirm the toolchain (Requirements 1–2): SharePoint Online Management Shell, PnP PowerShell,

Graph PowerShell.

1. Connect read-only to a pilot site:

```
Connect-PnPOnline -Url https://contoso.sharepoint.com/sites/Finance -Interactive
Get-PnPWeb | Select-Object Title, Url
```

1. For tenant-level reporting, also `Connect-SPOService -Url https://contoso-admin.sharepoint.com`

(Government: `https://contoso-admin.sharepoint.us` — **verify your cloud**).

A2. Run the discovery audit → `Permission Audit – SPO – Discovery`

1. Run `SPOPERM-Discovery.ps1` (read-only) against the pilot site(s). It inventories, per site:

• **Broken inheritance** — every web/list/library/folder/item with `HasUniqueRoleAssignments = true`.

• **Unique permissions** — the actual role assignments at each broken-inheritance scope.

• **Direct user grants** — role assignments whose principal is a **User** (not a SharePoint/Entra group).

• **Limited Access** — principals holding only the system-managed **Limited Access** level.

• **Over-share signals** — broad principals (e.g. **Everyone**, **Everyone except external users**, the

built-in **Members** group) granted **Edit/Full Control** at the site root.

```
.\SPOPERM-Discovery.ps1 -Url https://contoso.sharepoint.com/sites/Finance -CsvPath .\finance-audit.csv
```

1. Save the export as the baseline `Permission Audit – SPO – Discovery`. This is both the

before-state for rollback and the **exception list** the site owner reviews.

Gate A (pass/fail): You have a complete audit export for every pilot site, and the site owner has reviewed it and marked each unique-permission/direct-grant as **keep (documented exception)** or **remediate**. **Fail** = no baseline or no owner sign-off → do not proceed.

Stage B — Design: define the group + permission-level model

B1. Author `Access Model – SPO – Design`

Document, for each in-scope site:

1. **The groups that back access** — the three SharePoint groups and what feeds them:

• `SPO Group – Finance Owners – Prod` → **Full Control** (site owners only; keep tiny).

• `SPO Group – Finance Members – Prod` → **Edit** (contributors) — membership fed by the Entra/M365

group `M365 Group – Finance – Prod` where one exists.

• `SPO Group – Finance Visitors – Prod` → **Read** (read-only consumers).

1. **The permission-level model** — standardize on built-in **Full Control / Edit / Contribute / Read**

(see Overview). Define a custom level `Perm Level – Read No Download – Prod` *only* if a real view-only-no-download requirement exists. **Never edit built-in levels in place.**

1. **The unique-permission rule** — the default is **inherit**. A scope may keep unique permissions only

if it is (a) documented in the design, (b) owner-approved, and (c) still group-based (a *group* granted at the scope, never an individual). Everything else is remediated to inherit.

1. **Scopes:** name the pilot `Permission Scope – Pilot – Stage1` and the broad rollout

`Permission Scope – Broad – Stage2`.

Gate B: `Access Model – SPO – Design` is written and approved by the information owner and security stakeholder. **Fail** = no approved model → do not configure.

Stage C — Configure: stand up group-based access

C1. Ensure the site's SharePoint groups exist and carry the standard levels

1. For an **M365-group-connected Team site**, the group already provides Owners/Members; confirm the

SharePoint group associations and levels. For a **communication/classic site**, ensure the three SharePoint groups exist:

```
# Confirm the associated groups and their permission levels (read-only first)
Get-PnPGroup | Select-Object Title, Id
Get-PnPGroupPermissions -Identity "Finance Members"
```

1. Where a group is missing, create it and assign the standard level (dry-run first with the config

script, then `-Commit`):

```
.\SPOPERM-Remediate-GroupAccess.ps1 -Url https://contoso.sharepoint.com/sites/Finance -EnsureGroups -Wh
```

1. If you defined a custom level, create `Perm Level – Read No Download – Prod` by copying **Read** and

removing the download-related permissions in *Site permissions → Permission levels* (or via PnP role definition). **Verify the effective behavior in your tenant** before assigning it broadly.

C2. Feed group membership from the Entra/M365 source of truth

1. Where an Entra/M365 group is authoritative, make it the membership of the SharePoint Members group

(i.e. add `M365 Group – Finance – Prod` **as a member of** `SPO Group – Finance Members – Prod`, or use the M365 group directly for Team sites) so joins/leaves flow automatically.

1. Confirm no *individual users* are granted at the site root outside the three groups — those are the

direct grants Stage D remediates.

Gate C: The three named groups exist with the correct standard levels, and members flow from the governed group source where one exists. **Fail** = access still granted to individual users at the root → fix before remediating sub-scopes.

Stage D — Pilot remediation (`-WhatIf` first, then `-Commit`)

Do Stage D **only** on `Permission Scope – Pilot – Stage1`.

D1. Convert direct user grants → group membership (`Group Access Remediation – <Site> – Remediation`)

1. **Dry-run** the conversion — for each direct user grant flagged in the audit, the script proposes the

group the user should join (based on the granted level) and the direct grant it will remove:

```
.\SPOPERM-Remediate-GroupAccess.ps1 -Url https://contoso.sharepoint.com/sites/Finance `
    -ConvertDirectGrants -WhatIf
```

1. Review the proposal with the site owner. Where a direct grant maps to no legitimate group, the owner

decides the correct group (or that the access should be removed entirely).

1. **Commit** the approved conversions:

```
.\SPOPERM-Remediate-GroupAccess.ps1 -Url https://contoso.sharepoint.com/sites/Finance `
    -ConvertDirectGrants -Commit
```

This adds each user to the correct SharePoint group and removes the direct role assignment. Record it as `Group Access Remediation – Finance – Remediation`.

D2. Reset broken inheritance → inherit (`Inheritance Reset – <Site> – Remediation`)

1. **Dry-run** the reset — the script lists every scope it would reset to inherit, **excluding** the documented exceptions from `Access Model – SPO – Design`:

```
.\SPOPERM-Reset-Inheritance.ps1 -Url https://contoso.sharepoint.com/sites/Finance `
    -ExcludeList .\documented-exceptions.csv -WhatIf
```

1. Confirm the list matches the audit and the owner's keep/remediate decisions.
2. **Commit** the resets:

```
.\SPOPERM-Reset-Inheritance.ps1 -Url https://contoso.sharepoint.com/sites/Finance `
    -ExcludeList .\documented-exceptions.csv -Commit
```

Each reset calls the scope's `ResetRoleInheritance()`, re-inheriting site permissions and removing the unique grant (and any **Limited Access** it created). Record as `Inheritance Reset – Finance – Remediation`.

> ***Rollback.*** *If a reset removes access someone genuinely needed, re-grant it *as a group* at the minimum necessary scope, or (last resort) re-break inheritance and re-add the documented grant, using the `Permission Audit – SPO – Discovery` baseline as the record of the prior state.*

Stage E — Validate the pilot

Run the validation script and the tests in **05-testing.md** against `Permission Scope – Pilot – Stage1`:

```
.\SPOPERM-AccessReview.ps1 -Url https://contoso.sharepoint.com/sites/Finance -Report -CsvPath .\finance-ac
```

Gate E (pass/fail):

- Access at the site is **group-based** — no individual users granted directly at the root (system accounts excepted).
- **Broken inheritance count is reduced** to only the documented exceptions.
- **Limited Access sprawl is reduced** correspondingly (it disappears when the underlying unique grant is reset).
- **No over-share** — no broad principal holds Edit/Full Control at the root unintentionally.
- A **re-run audit** matches the intended end-state; the owner confirms no one lost needed access.

Fail any item → remediate or roll back before expanding.

Stage F — Stand up access reviews, then expand

F1. Wire the recurring review → `Access Review – <Site> – Prod`

- **If Entra ID P2 is licensed:** create a **Microsoft Entra ID Access Review** over the membership of

`M365 Group – Finance – Prod` (and/or the site owners), recurring (e.g. quarterly), with the site owner as reviewer and "remove access if not reviewed" per the org's policy. Name it `Access Review – Finance – Prod`. *(Entra admin center → Identity Governance → Access reviews — verify the exact path in your tenant.)*

- **If SharePoint Advanced Management (SAM) is licensed:** additionally enable **site access review** and

the **Data Access Governance** reports in the SharePoint admin center to catch over-sharing between reviews.

- **If neither add-on is licensed:** schedule the included `SPOPERM-AccessReview.ps1 -Report` on a

recurring basis and route the export to the owner for attestation. Same evidence, no add-on.

F2. Establish the recurring access report → `Access Report – SPO – Report`

- Schedule `SPOPERM-AccessReview.ps1 -Report` across in-scope sites to produce a repeatable

who-has-access export, and (where SAM is licensed) supplement with the DAG reports. Store exports with the review records.

F3. Expand to `Permission Scope – Broad – Stage2`

- Repeat Stages A–E per site (or in owner-approved batches) across the broad scope, always: fresh audit →

owner review → `-WhatIf` → `-Commit` → validate. Never expand past a failed gate.

Gate F: A recurring review exists for each in-scope site (via P2 Access Reviews, SAM site review, or the scheduled script), and the recurring `Access Report – SPO – Report` is running. **Fail** = no ongoing attestation → the model will drift; close before declaring done.

5. Testing & Validation

Run these after the pilot remediation (Stage D) and again after each expansion batch. Each test states the **procedure** and the **expected pass/fail signal**. Run read-only checks with `SPOPERM-Discovery.ps1` and `SPOPERM-AccessReview.ps1`; do not use `-Commit` here. Substitute your tenant/site for the `contoso.sharepoint.com` / `Finance` examples.

Test 1 — Access is group-based at the site root

- **Procedure:** Re-run the audit and inspect the root web's role assignments:

```
.\SPOPERM-Discovery.ps1 -Url https://contoso.sharepoint.com/sites/Finance -CsvPath .\finance-audit-post.
```

- **Pass:** Every root role assignment is a **SharePoint group** (Owners/Members/Visitors) or a governed

Entra/M365 group — no individual **User** principals (system accounts like the site collection admin excepted).

- **Fail:** Any individual user still granted directly at the root → convert them via

`Group Access Remediation – Finance – Remediation`.

Test 2 — Standard permission levels are correctly bound

- **Procedure:** `Get-PnPGroupPermissions -Identity "Finance Owners"` (and Members/Visitors).
- **Pass:** `SPO Group – Finance Owners – Prod` = **Full Control**, `...Members...` = **Edit**, `...Visitors...` = **Read**. Any custom level in use is exactly `Perm Level – Read No Download – Prod`, and **built-in levels are unmodified**.
- **Fail:** A group carries the wrong level, or a built-in level was edited in place → correct the binding / recreate the built-in level.

Test 3 — Broken inheritance reduced to documented exceptions only

- **Procedure:** From the post-remediation audit, count scopes with `HasUniqueRoleAssignments = true` and compare against `documented-exceptions.csv` (from `Access Model – SPO – Design`).
- **Pass:** The only scopes with unique permissions are the **documented, owner-approved exceptions**; their unique grants are **groups**, not individuals.
- **Fail:** Undocumented broken-inheritance scopes remain → reset them (`Inheritance Reset – Finance – Remediation`), or add to the design if genuinely required (owner-approved).

Test 4 — Limited Access sprawl reduced

- **Procedure:** In the post-remediation audit, count principals whose only level is **Limited Access**.
- **Pass:** Limited Access entries dropped in step with the inheritance resets (Limited Access disappears when the underlying sub-scope unique grant is removed).
- **Fail:** Limited Access persists with no corresponding remaining unique grant → find and reset the sub-scope (list/folder/item) still holding a unique permission for that principal.

Test 5 — No unintended over-share

- **Procedure:** In the audit, check for broad principals — **Everyone**, **Everyone except external users**, **the site** Members **group — holding** Edit **or** Full Control** at the root or on sensitive libraries.
- **Pass:** No broad principal holds Edit/Full Control unintentionally; broad read access, if present, is intentional and owner-approved.
- **Fail:** A broad principal has more than intended → reduce the level or remove the principal per the owner's decision.

Test 6 — A converted user actually has the right effective access

• **Procedure:** Pick a user converted in Stage D. Confirm group membership

(`Get-PnPGroupMember -Identity "Finance Members"`), then verify effective access — e.g. `Get-PnPUserEffectivePermissions` for the user on the site, or have the user confirm they can still open the content they need (and nothing they shouldn't).

• **Pass:** The user's effective permissions match the group's level; they retain intended access via the group, with no leftover direct grant.

• **Fail:** Effective access is wrong (too much/too little) → correct group membership; ensure the old direct grant was removed.

Test 7 — Access review is running

• **Procedure:** Confirm the recurring review exists — the Entra ID Access Review `Access Review – Finance – Prod (if P2), the SharePoint site access review (if SAM), or the scheduled SPOPERM-AccessReview.ps1 -Report` job — and that it produced its first output.

• **Pass:** A recurring review/report is configured with the site owner as reviewer/recipient and has run at least once.

• **Fail:** No recurring attestation exists → wire it (Stage F) before declaring done.

Test 8 — The access report reproduces cleanly

• **Procedure:** Run the reporting path end-to-end:

```
.\SPOPERM-AccessReview.ps1 -Url https://contoso.sharepoint.com/sites/Finance -Report -CsvPath .\finance-
```

• **Pass:** `Access Report – SPO – Report` produces a per-principal who-has-access export (group, expanded members, level, scope) without errors, suitable for owner attestation.

• **Fail:** The report errors or omits expected principals → check Graph/PnP scopes (Requirements 2, 6) and group-expansion permissions.

Test 9 — Rollback works (pilot only, non-destructive rehearsal)

• **Procedure:** On a **non-production test scope**, confirm you can restore a prior unique grant from the `Permission Audit – SPO – Discovery` baseline (re-add the documented group at the scope).

• **Pass:** The prior state is restorable from the baseline export; the rollback procedure is understood before any broad-scope `-Commit`.

• **Fail:** No usable before-state → re-capture the audit baseline before proceeding.

Overall gate: Tests 1–5 and 7–8 **pass** on `Permission Scope – Pilot – Stage1`, Test 6 confirms real users are unaffected, and Test 9's rollback is understood — before expanding to `Permission Scope – Broad – Stage2`.

6. Troubleshooting

Common failure modes and how to resolve them. Symptoms reference the artifacts and scripts from the other sections. Work read-only first; never `-Commit` while diagnosing.

Connection & permission problems

`Connect-PnPOnline -Interactive` *fails or prompts for an app*

- **Cause:** PnP.PowerShell v2+ requires an **Entra app registration** for interactive sign-in; without

one, the connection is rejected.

- **Fix:** Register access once (`Register-PnPManagementShellAccess`) or use your org's PnP app, and

consent the delegated SharePoint permissions (Requirement 2). Re-run `SPOPERM-Scope.ps1` to confirm.

Scripts connect but can't read groups / role assignments ("access denied")

- **Cause:** The account is not **Site Collection Admin** on the target site (SharePoint Admin alone does

not grant per-site object access until it's SCA).

- **Fix:** Add the account as SCA (SharePoint admin center → *Sites → Active sites → <site> → Membership*,

or `Set-SPOUser -Site <url> -LoginName <upn> -IsSiteCollectionAdmin $true`). `SPOPERM-Scope.ps1` reports this as the "group-read (SCA proxy)" check.

Government cloud: URLs/cmdlets don't resolve

- **Cause:** Commercial URLs/endpoints were used on **GCC High / DoD**.

- **Fix:** Use your cloud's admin/site URLs (`*.sharepoint.us`) and connect Graph with the right

environment (`Connect-MgGraph -Environment USGov` / `USGovDoD`). **Verify the exact paths in your tenant** — do not assume the commercial ones.

Audit / discovery problems

`SPOPERM-Discovery.ps1` *reports "no unique permissions" but you know sharing happened*

- **Cause 1:** The unique permissions are at **item/folder** level, and you ran without `-IncludeItems`

(which only scans the web + lists by default for performance).

- **Fix 1:** Re-run with `-IncludeItems` on the suspect library. Expect it to be slower.

- **Cause 2:** Access came via a **sharing link** (which grants through a system group) rather than a

classic unique role assignment — that is **external/link sharing**, a different product's scope (see Overview → Scope). This solution governs the internal permission model, not link sharing.

The audit shows a broad principal (e.g. "Everyone except external users") — is that always bad?

• **Cause:** Broad principals are only a problem at **high levels** (Edit/Full Control) or on sensitive content. Broad **Read** on genuinely all-hands content may be intentional.

• **Fix:** Treat `OverShare = True` rows (broad principal **and** a high level) as findings for the owner; confirm broad Read is intentional before removing it.

Remediation problems

`SPOPERM-Remediate-GroupAccess.ps1 -ConvertDirectGrants` *skips a user ("no standard group maps")*

• **Cause:** The user's direct grant is a **custom or unusual level** (e.g. only `Perm Level – Read No Download – Prod`, or a bespoke level) that doesn't map to Owners/Members/Visitors automatically.

• **Fix:** The **information owner decides** the correct group (or that access should be removed). Add the user to that group manually, then remove the direct grant — this is exactly the owner-decision the script defers rather than guessing.

`SPOPERM-Reset-Inheritance.ps1` *won't reset a scope / reset "succeeds" but it's still unique*

• **Cause 1:** The scope is in the `-ExcludeList` of documented exceptions (by design — check the KEEP lines).

• **Cause 2:** A **child** scope below it still has its own unique permissions; resetting the parent does not touch independently-broken children.

• **Fix:** Confirm the scope isn't an intended exception; if a child is still unique, reset it too (run with `-IncludeItems` to reach item/folder scopes). Re-run `SPOPERM-Discovery.ps1` to confirm.

After a reset, someone lost access they actually needed

• **Cause:** A unique grant was **load-bearing but undocumented** — exactly the risk the pilot/`-WhatIf` flow exists to catch.

• **Fix (rollback):** Using the `Permission Audit – SPO – Discovery` baseline as the record of prior state, re-grant the access **as a group** at the minimum necessary scope (preferred), or restore the documented unique grant. Then add the scope to the design's exception list so future runs keep it.

Limited Access & inheritance quirks

"Limited Access" won't go away when I try to remove it

• **Cause: Limited Access is system-managed** — you cannot assign or delete it directly. It exists because a **child** list/folder/item still has a unique grant for that principal.

• **Fix:** Find the underlying sub-scope still granting that principal (audit with `-IncludeItems`) and

reset **it** to inherit; Limited Access at the parent disappears once its cause is gone.

Broken-inheritance count didn't drop as much as expected

• **Cause:** Item/folder-level scopes weren't included in the reset run.

• **Fix:** Re-run `SPOPERM-Reset-Inheritance.ps1 -IncludeItems` on the affected library (still `-WhatIf`

first). Very large libraries take time; scope deliberately.

Ongoing model / drift problems

Direct user grants keep reappearing after remediation

• **Cause:** Someone is "just adding a user" to resolve urgent access requests — the classic regression.

• **Fix:** Enforce the group-only rule (Best Practices), and run the recurring `Access Report – SPO –

Report / SPOPERM-AccessReview.ps1 -Report` — its **"direct-grant leftovers"** count surfaces new direct grants so they're converted quickly. Coach owners to add people to the group, not the site.

Can't create an Entra ID Access Review / no Data Access Governance reports in the admin center

• **Cause: Access Reviews require Entra ID P2**; **DAG reports / site access review require SharePoint

Advanced Management (SAM)**. If unlicensed, the features simply don't appear.

• **Fix:** Either license the add-on under change control, **or** use the included

`SPOPERM-AccessReview.ps1 -Report` on a schedule for the same attestation evidence. **Verify licensing in your tenant** — availability is plan-dependent and changes over time.

The access report is missing members of a group

• **Cause:** The group is a **nested Entra/M365 group** and the report ran without `-ExpandEntraGroups`,

or the Graph read scopes weren't consented.

• **Fix:** Re-run `SPOPERM-AccessReview.ps1 -Report -ExpandEntraGroups` and ensure delegated

`Group.Read.All` / `Directory.Read.All` are consented (Requirement 6).

A custom `Perm Level – Read No Download – Prod` *isn't actually blocking downloads*

• **Cause:** Permission-level mechanics are nuanced — removing certain permissions doesn't always prevent

every copy/download path (and Office Online "View Only" behaves differently from a custom Read level).

• **Fix: Verify the effective behavior in your tenant** before relying on it; for hard

download-prevention consider the appropriate protection control rather than a permission level alone. Never edit the **built-in** Read level to achieve this.

7. Best Practices

Durable practices for keeping SharePoint access intentional and auditable after the initial remediation.

Model & structure

- **Grant to groups, never to users.** Make it a hard rule: a person never appears in a role assignment;

a **group** does. Add the person to `SPO Group – Finance Members – Prod` (or the backing `M365 Group – Finance – Prod`), not to the site or the item. Group-based access is the single most important habit — it is what makes access legible and self-cleaning.

- **Back SharePoint groups with a governed Entra/M365 group** wherever possible, so joins/leaves/transfers

flow automatically from the identity source of truth rather than being maintained by hand.

- **Keep the Owners group tiny.** Full Control includes permission management; every extra owner is

someone who can re-break the model. Two or three owners per site is plenty.

- **Standardize on the built-in permission levels** (Full Control / Edit / Contribute / Read). Introduce a

custom level (`Perm Level – Read No Download – Prod`) only for a real requirement the built-ins can't express, and **never edit a built-in level in place** — it changes behavior everywhere it's used.

Inheritance & unique permissions

- **Inheritance is the default; unique permissions are an exception.** Every place inheritance is broken

is a boundary you must maintain forever. Treat each one as debt: it needs a documented reason, an owner, and a group (not a user) grant — or it gets reset to inherit.

- **Don't break inheritance to solve a one-off.** The reflex "just give them this one folder" is how the

tangle grows. Prefer a **separate site or library** with its own group for content that genuinely needs a different audience, rather than a unique-permission scope buried inside a shared site.

- **Remediate Limited Access by removing its cause.** Limited Access is system-managed noise created by

sub-scope unique grants; you clear it by resetting the underlying scope, not by touching Limited Access.

- **Prefer library/list-level boundaries over item-level.** Item-level unique permissions are the hardest

to audit and the easiest to orphan. If a boundary is needed, put it as high as possible.

Change discipline

- **Audit before you change, always.** The `Permission Audit – SPO – Discovery` export is both your

evidence and your rollback baseline. Never remediate without a current one.

- `-WhatIf` **before** `-Commit`**, pilot before broad.** High-impact permission changes go through the full

house cycle every time: read-only audit → owner review → dry-run → commit → validate.

- **Owner-approve every exception.** A unique-permission scope survives only if the information owner signs

off; otherwise it's remediated. Keep the exception list in `Access Model – SPO – Design` current.

- **Name everything consistently.** Reuse the `<Purpose> – <Scope> – <Stage>` names across groups,

reviews, reports, and change records so the model is self-documenting.

Ongoing attestation & drift control

- **Run recurring access reviews.** Access is only as good as its last review. Use Entra ID Access Reviews

(P2) for group membership, SharePoint site access review + Data Access Governance reports (SAM) for over-share detection, or the scheduled `SPOPERM-AccessReview.ps1 -Report` where the add-ons aren't licensed — whichever the tenant supports.

- **Report on a schedule, not just on request.** A recurring `Access Report – SPO – Report` catches drift

(new direct grants, new broken inheritance) between formal reviews.

- **Re-baseline after big changes.** After a reorg, migration, or mass onboarding, re-run discovery and

compare — new sprawl is easiest to fix while it's small.

- **Watch for the return of direct grants.** The failure mode is always the same: someone "just adds a

user" to fix an urgent access request. Make the recurring report flag new direct grants so they're converted to group membership quickly.

Cloud & licensing awareness

- **Verify plan-dependent features in your tenant.** Access Reviews (Entra ID P2) and DAG reports / site

access review (SAM) are add-on-gated and evolve — confirm availability before designing around them, and keep the PowerShell fallback path viable.

- **Mind Commercial vs Government cloud differences.** Admin-center URLs and some feature availability

differ on GCC / GCC High / DoD (`*.sharepoint.us`, `portal.azure.us`) — verify paths on your cloud rather than assuming the commercial ones.

8. Deliverables Summary

Everything included in this solution, on one page.

Documentation

Document	What it gives you
1. Solution Overview	What the permission/access model is, how it works, the standard permission-level model, the reference artifacts, benefits, limitations, and the boundaries vs external sharing / IA / records.
2. Requirements	Solution-specific prerequisites (SharePoint Admin + SCA, PnP app registration, group source of truth, information owners, P2/SAM licensing, Graph read scopes, pilot + rollback) with verify/implement steps — on top of the shared SharePoint global requirements.
3. Implementation	Exact ordered steps: assess (audit) → design the group + level model → configure group-based access → remediate direct grants + broken inheritance (pilot, `-WhatIf` first) → validate → stand up reviews → expand. Every artifact named; pass/fail gates and rollback throughout.
5. Testing & Validation	Nine tests (group-based root, correct levels, inheritance reduced to exceptions, Limited Access reduced, no over-share, converted-user effective access, review running, report reproduces, rollback understood), each with pass/fail.
6. Troubleshooting	Common failure modes (audit shows nothing, can't reset inheritance, Limited Access persists, direct grants reappear, review/report gaps, Gov-cloud paths) with root cause and fix.
7. Best Practices	Durable practices for group-based access, inheritance discipline, change control, ongoing attestation, and cloud/licensing awareness.
Executive Summary	Crisp outcome/approach/why-low-risk summary for decision-makers.

PowerShell scripts (5) — SharePoint Online Management Shell / PnP PowerShell / Microsoft Graph

All are **safe by default** (read-only, or `-WhatIf` / dry-run unless an explicit `-Commit`), use modern-auth `Connect-PnPOnline` / `Connect-MgGraph` (no stored credentials), and reuse the artifact names from the reference model.

Script	What it does	Safe default
SPOPERM-Scope.ps1	Pre-flight scope/health check: modules installed, sign-in works, group-read (Site Collection Admin proxy), standard groups/levels resolve, and the broken-inheritance footprint sized. Run first.	Read-only
SPOPERM-Discovery.ps1	Read-only audit → `Permission Audit – SPO – Discovery`: broken inheritance, unique permissions, **direct user grants**, **Limited Access**, and **over-share** signals per site. The rollback baseline + owner exception list.	Read-only
SPOPERM-Remediate-GroupAccess.ps1	Moves the site to **group-based access**: ensures the standard Owners/Members/Visitors groups + levels exist, and converts **direct user grants** into group membership. Records `Group Access Remediation – <Site> – Remediation`.	Dry-run unless `-Commit`; `-WhatIf`
SPOPERM-Reset-Inheritance.ps1	Resets **broken-inheritance** scopes back to inherit (clearing unique permissions and the Limited Access they created), **excluding** documented exceptions. Records `Inheritance Reset – <Site> – Remediation`.	Dry-run unless `-Commit`; `-WhatIf`
SPOPERM-AccessReview.ps1	Read-only who-has-access report → `Access Report – SPO – Report`: SharePoint groups expanded to members, including nested Entra/M365 groups via Graph. The recurring attestation evidence (`Access Review – <Site> – Prod`) when P2/SAM features aren't licensed.	Read-only

Licensing note: Entra ID Access Reviews *(P2) and SharePoint* ***Data Access Governance reports / site access review*** *(SharePoint Advanced Management) are* ***plan-dependent add-ons*** *— verify in your tenant/plan. Where they aren't licensed, the included PowerShell reporting provides the same evidence.* ***Who belongs in each group is an information-owner decision****, not an implementer one.*

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

SPOPERM-AccessReview.ps1

SPOPERM-Discovery.ps1

SPOPERM-Remediate-GroupAccess.ps1

SPOPERM-Reset-Inheritance.ps1

SPOPERM-Scope.ps1